Nicholas Guevara & William Dappen
2/28/26

# CS 415 Project 1A

# Asymptotic Analysis of mélange of Algorithms

Project 1A

Introduction:

The goal of this project was to measure and compare the asymptotic growth rates of the basic operations of several algorithms, including Fibonacci, Euclid's GCD, exponentiation, and sorting algorithms. For each algorithm, we specified a basic operation and designed an implementation to count the number of times that operation was executed as input sizes increased. These counts were plotted using scatter plots to examine their growth and compare it to theoretical analysis.

Task 1: Fibonacci and GCD

First, we analyzed the recursive Fibonacci Algorithm. We counted the number of additions done by the algorithm, shown as A(k), while computing Fib(k). The upper bound of the implementation is actually k = 91, which was chosen because the algorithm utilizes fib(k+1) and fib(93) overflows a 64-bit integer. For the purpose of the scatter plot display we limited it to k = 40 for readability. We expected exponential growth based on our previous analysis of the recurrence relation, and the scatter plot confirms this. This plot uses a logarithmic scale on the y-axis, A(k), to show the relationship as clearly as possible. The scatter plot shows likely asymptotic complexity of $\Theta(\varphi^k)$ which is what our in-class analysis concluded.

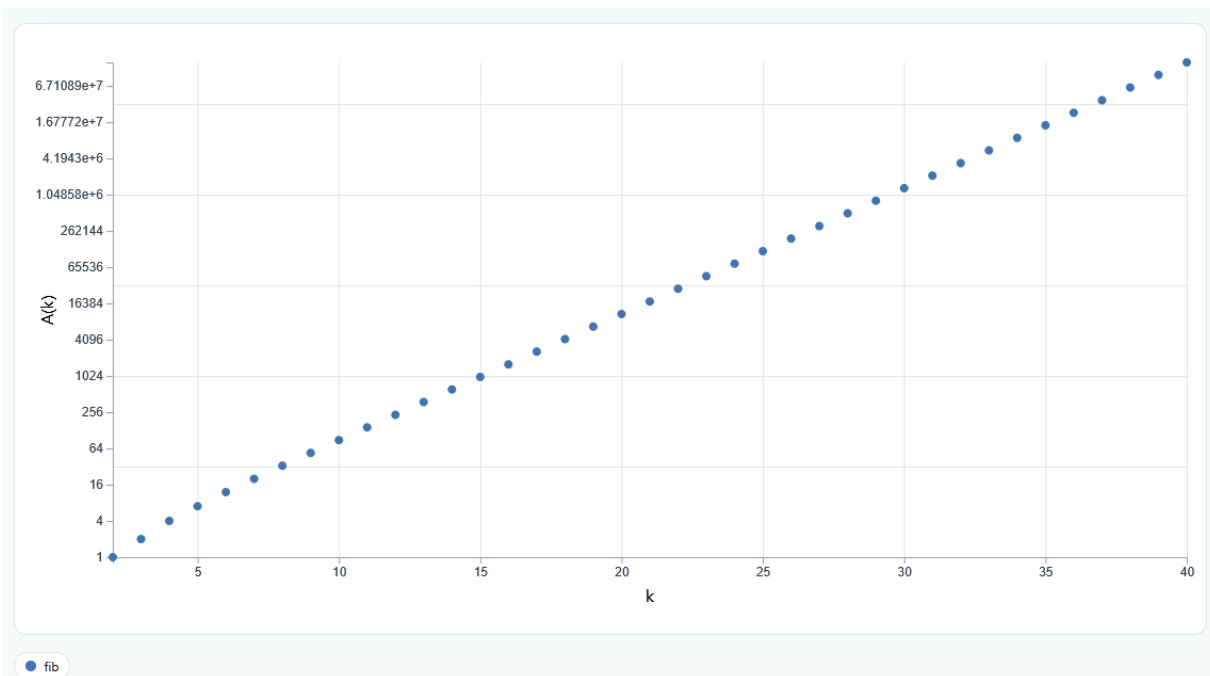


Fig. 1: Scatter plot of the number of additions A(k) employed by the recursive Fibonacci algorithm for values of k up to 40.

Next, we analyzed Euclid's algorithm for computing GCD. The basic operation in this algorithm is modulo division. We counted these divisions as D(n) during computation of the algorithm under a worst-case scenario with inputs which are consecutive elements of the fibonacci sequence derived from the previous section. Each n on the plot is the smaller Fibonacci number of the pair.

From the plot it appears the growth of D(n) is logarithmic as n increases. The input values grow exponentially, but the number of divisions scales linearly with the "index" of the input, which previously was referred to as k. From this data, Θ(log n) is the likely complexity, where n is the fibonacci number used as the input.

Fig. 2: Scatter plot of the number of modulo divisions, D(n) performed by Euclid's algorithm against worst-case inputs n, consecutive elements of the Fibonacci sequence.

Task 2: Exponentiation

We measured three different exponentiation algorithms by counting the number of multiplications required as the exponent (n) increased, represented on the scatter plot as M(n). The decrease-by-one algorithm exhibits linear growth, consistent with Θ(n) time complexity, as shown by its approximately straight-line behavior in the graph. The decrease-and-conquer algorithm shows logarithmic growth, consistent with Θ(log n). The third algorithm also has overall Θ(n) complexity, but its curve displays noticeable stairstep increases. These steps occur because the algorithm makes two recursive calls on the floor of n/2, forming a binary recursion tree. Whenever the exponent crosses a power of two (most noticeably 1024), the recursion tree gains an additional level of depth. This effectively doubles the number of leaf computations at that level, producing the sharp upward jumps visible in the plot. Between these powers of two, the growth is approximately linear, but each new level of recursion introduces a discrete increase in total multiplications, creating the staircase pattern. We chose a limit n = 2000 to show this pattern clearly, while not being so large as to overwhelm the graph.

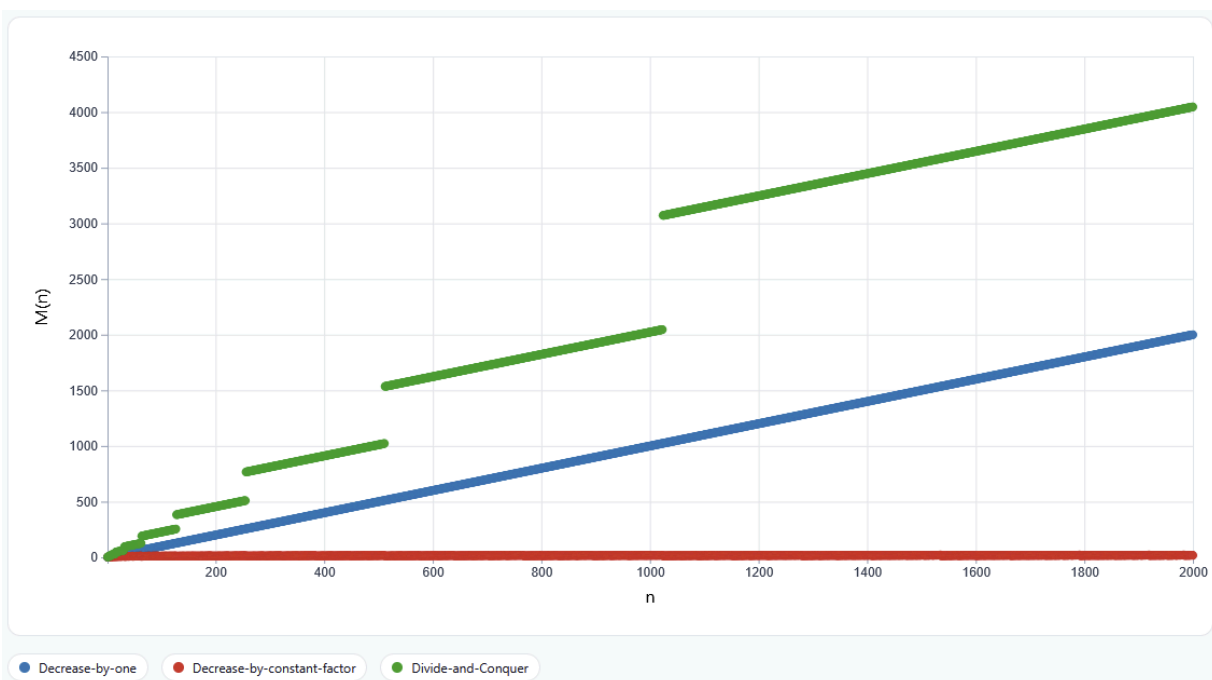


Fig. 3: A scatter plot of three, exponentiation functions, Decrease by One, Decrease by Constant Factor, and Divide and Conquer. M(n) is the number of multiplications performed by the algorithm for various inputs n.

Task 3: Insertion Sort vs. Selection Sort

Finally, we compared insertion sort and selection sort by measuring the number of comparisons C(n) performed by the algorithms for three scenarios: best-case inputs of a sorted list, average-case inputs using a random list, and worst-case inputs using a reverse-sorted list. We scatter plotted the comparisons against n, the number of elements in the arrays to be sorted.

In the best-case scenario, insertion sort shows linear growth, consistent with $\Theta(n)$ complexity. Since the array is ordered, insertion sort can finish in a single pass.The selection sort appears to grow quadratically with $\Theta(n^2)$ complexity, since it needs to scan the entire array every pass regardless of the state of the array. This matches what we would expect in this situation.

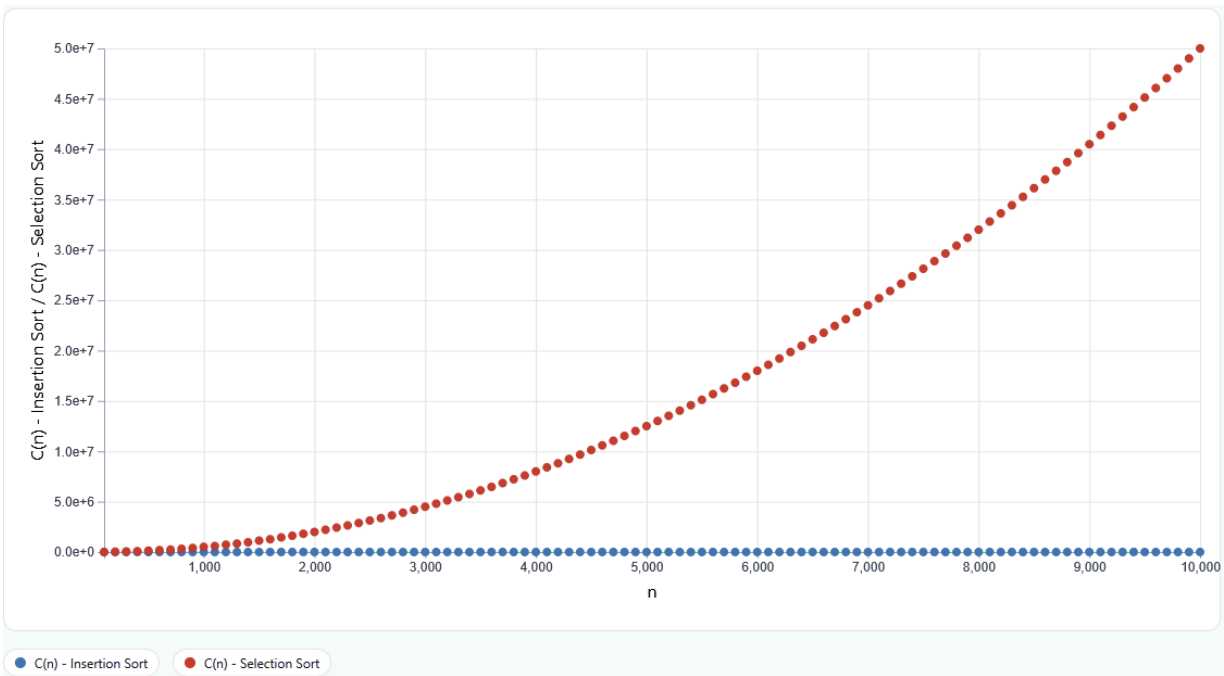


Fig. 4: A graph of the comparisons C(n) done by the two sorts on arrays of size n in a best-case scenario.

In the average case, both algorithms show growth consistent with $\Theta(n^2)$. However even in this case insertion sort outperforms by a constant factor, because it will not need to scan the entire array each pass, again matching our theoretical expectation.

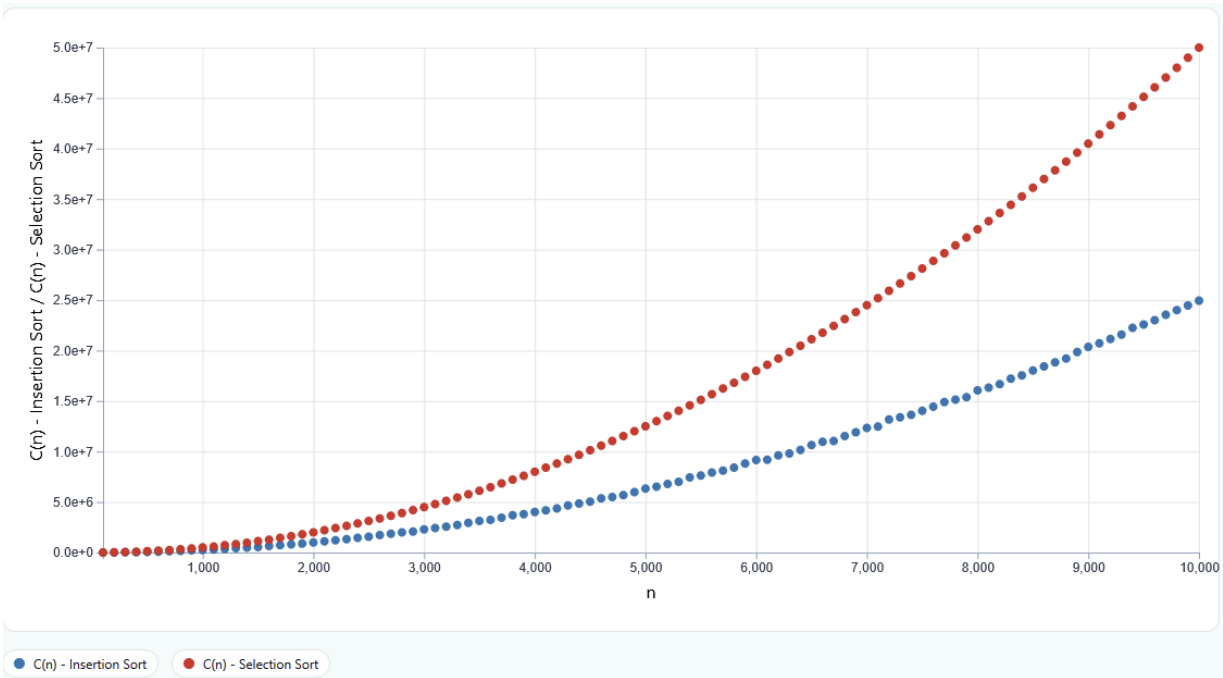


Fig. 5: A graph of the comparisons C(n) done by the two sorts on arrays of size n in an average-case scenario.

In the worst case scenario, both algorithms demonstrate quadratic growth consistent with $\Theta(n^2)$. The two plots are exactly the same because both algorithms must scan the entire array each pass, so the advantages of the insertion sort are nullified.

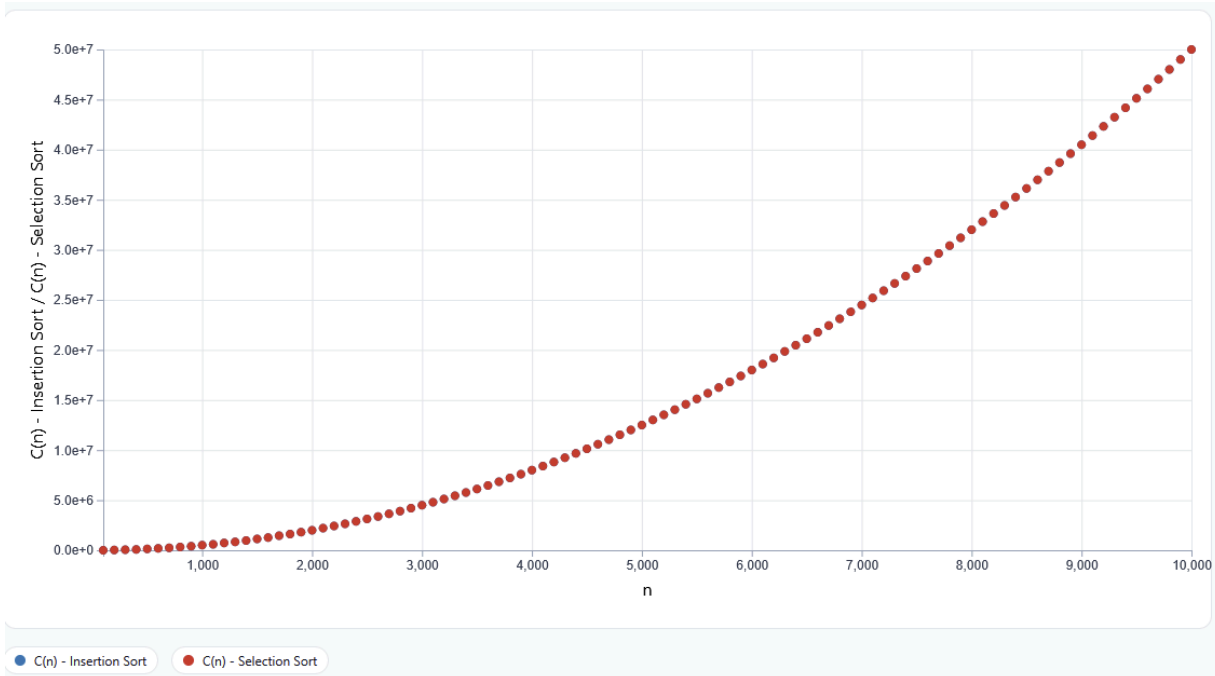


Fig. 6: A scatter plot of the comparisons C(n) done by the two sorts on arrays of size n in a worst-case scenario. The two plots are perfectly overlaid, so only the red line is visible.